

Package ‘CohortPrevalence’

September 24, 2026

Title Standardized Prevalence Calculator for OMOP/OHDSI Ecosystem

Version 1.2.1

Description This package calculates prevalence of a condition in a population.

License Apache License

Encoding UTF-8

Roxygen list(markdown = TRUE)

Depends R (>= 4.1.0)

Imports checkmate,
cli,
crayon,
digest,
dplyr,
fs,
glue,
here,
jsonlite,
lubridate,
methods,
purrr,
R6,
readr,
rlang,
snakecase,
SqlRender,
DatabaseConnector,
tibble,
tidyr

Additional_repositories <https://OHDSI.github.io/drat>

Suggests knitr,
plotly,
reactable,
rmarkdown,
shiny,
testthat (>= 3.0.0)

VignetteBuilder knitr

Config/testthat/edition 3

LazyData true

Config/roxygen2/version 8.0.0

RoxygenNote 7.3.2

R topics documented:

acs_age_groups	2
CohortPrevalenceExperiment	3
createCohortPrevalenceAnalysis	6
createDemographicConstraints	8
createDenominatorType	8
createPopulationCohort	9
createPrevalenceType	9
createRassenIncidenceAnalysis	10
createSpan	11
createStandardizationReference	12
createTargetCohort	13
createYearlyRange	13
exportPrevalenceQuery	14
fetchACSReference	14
generatePrevalence	15
getStandardizationReference	16
importStandardizationReference	17
listStandardizationReferences	18
loadPrevalenceResults	19
map_acs_b01001_to_age_sex	19
PopulationCohort	20
PrevalenceResults	21
StandardizationReference	24
TargetCohort	27

Index	29
--------------	-----------

acs_age_groups	<i>ACS B01001 Age Group Reference Table</i>
----------------	---

Description

Single canonical table for the 23 ACS B01001 age-sex groups. Each consumer (API variable mapping, single-year age range lookup, etc.) selects only the columns it needs.

Usage

```
acs_age_groups()
```

Details

Replaces the old `get_b01001_mapping()`. The 23 group labels appear in exactly one place.

Value

Data frame with one row per age group and columns:

age_label Age group label (e.g., "0-4", "5-9", ..., "85+")

min_age Minimum single-year age in the group (inclusive)

max_age Maximum single-year age in the group (inclusive; Inf for open-ended)

male_variable B01001 variable code for males (e.g., "B01001_003")

female_variable B01001 variable code for females (e.g., "B01001_027")

CohortPrevalenceExperiment

CohortPrevalenceExperiment R6 Class

Description

Define and manage multi-dimensional prevalence analysis experiments. Enables users to specify cohorts, prevalence types, demographic constraints, and periods of interest, then automatically generates a Cartesian product of analysis definitions with full provenance tracking.

Details

The experiment class uses the builder pattern to define analysis dimensions:

1. Add cohorts (one or more)
2. Add prevalence types (one or more)
3. Add demographic constraints (one or more)
4. Add periods of interest (one or more)
5. Set common parameters (strata, output types)
6. Call `define()` to materialize analysis objects
7. Pass analysis list to `generatePrevalence()`

Design Philosophy:

- **Input:** Object-based (type-safe), validated on each addition
- **Output:** Flat tibble specification, CSV-exportable for reproducibility
- **Execution:** `exp$define()` → `generatePrevalence(analyses, ...)`
 - Explicit data flow: `experiment` defines dimensions, `define()` materializes analyses, `generatePrevalence()` executes

Example:

```
exp <- CohortPrevalenceExperiment$new("CKD Multi-Cohort")

exp$addCohorts(tibble::tibble(
  cohortId = c(1, 2, 3),
  cohortName = c("CKD A", "CKD B", "CKD C")
))

exp$addPrevalenceTypes(list(
```

```

    createPrevalenceType("point", lookBackDays = 0L),
    createPrevalenceType("period_prevalence_pd2", lookBackDays = 365L, leadInDays = 365L),
    createPrevalenceType("period_prevalence_pd3", lookBackDays = 0L)
  ))

exp$addDemographicConstraints(list(
  createDemographicConstraints(ageMin = 18, ageMax = 150, genderIds = c(8507L, 8532L, 0L)),
  createDemographicConstraints(ageMin = 65, ageMax = 150, genderIds = c(8507L, 8532L, 0L)),
  createDemographicConstraints(ageMin = 18, ageMax = 150, genderIds = c(8507L))
))

exp$addPeriodsOfInterest(list(
  createYearlyRange(2015:2024),
  createSpan(start = "2020-01-01", end = "2020-12-31"),
  createSpan(start = "2021-01-01", end = "2021-12-31")
))

exp$setCommonParameters(strata = c("age", "gender"), outputTypes = "prevalence")

# View specification
exp$viewDesign()
exp$viewDesign("html")

# Define analyses (3 × 3 × 3 × 3 = 81 analyses)
analyses <- exp$define()

# Execute
results <- generatePrevalence(analyses, executionSettings = settings)

```

Methods

Public methods:

- `CohortPrevalenceExperiment$new()`
- `CohortPrevalenceExperiment$addCohorts()`
- `CohortPrevalenceExperiment$addPrevalenceTypes()`
- `CohortPrevalenceExperiment$addDemographicConstraints()`
- `CohortPrevalenceExperiment$addPeriodsOfInterest()`
- `CohortPrevalenceExperiment$setCommonParameters()`
- `CohortPrevalenceExperiment$validate()`
- `CohortPrevalenceExperiment$getSpecification()`
- `CohortPrevalenceExperiment$viewDesign()`
- `CohortPrevalenceExperiment$define()`
- `CohortPrevalenceExperiment$clone()`

Method `new()`: Create new experiment

Usage:

```
CohortPrevalenceExperiment$new(name, description = NULL)
```

Arguments:

`name` Character. Descriptive name for the experiment

`description` Character. Optional description

Method addCohorts(): Add cohort definitions

Usage:

```
CohortPrevalenceExperiment$addCohorts(cohorts)
```

Arguments:

cohorts Tibble with columns:

- cohortId (numeric, required): cohort ID in the results schema
- cohortName (character, required): display name

Returns: Invisibly returns self for method chaining

Method addPrevalenceTypes(): Add prevalence type definitions

Usage:

```
CohortPrevalenceExperiment$addPrevalenceTypes(types_list)
```

Arguments:

types_list List of prevalenceType objects (created via createPrevalenceType)

Returns: Invisibly returns self for method chaining

Method addDemographicConstraints(): Add demographic constraint definitions

Usage:

```
CohortPrevalenceExperiment$addDemographicConstraints(constraints_list)
```

Arguments:

constraints_list List of demographicConstraints objects (created via createDemographicConstraints)

Returns: Invisibly returns self for method chaining

Method addPeriodsOfInterest(): Add periods of interest

Usage:

```
CohortPrevalenceExperiment$addPeriodsOfInterest(poi_list)
```

Arguments:

poi_list List of period objects (YearlyRange or Span objects)

Returns: Invisibly returns self for method chaining

Method setCommonParameters(): Set common parameters for all analyses

Usage:

```
CohortPrevalenceExperiment$setCommonParameters(
  strata = NULL,
  outputTypes = NULL,
  useOnlyFirstObservationPeriod = FALSE,
  minimumObservationLength = NULL
)
```

Arguments:

strata Character vector of strata variables (e.g., c("age", "gender"))

outputTypes Character vector of output types (e.g., "prevalence")

useOnlyFirstObservationPeriod Logical. If TRUE, only first observation period per person is used

minimumObservationLength Removed. Lead-in days are now set per prevalence type via createPrevalenceType(leadInDays =).

Returns: Invisibly returns self for method chaining

Method validate(): Validate all required parameters are set

Usage:

CohortPrevalenceExperiment\$validate()

Returns: Invisibly returns TRUE if valid, otherwise stops with error

Method getSpecification(): Get specification as flat tibble

Usage:

CohortPrevalenceExperiment\$getSpecification()

Returns: Tibble with one row per analysis combination

Method viewDesign(): View experiment design

Usage:

CohortPrevalenceExperiment\$viewDesign(format = "console")

Arguments:

format Character. "console" (default) for text table, "html" for interactive reactable

Returns: Invisibly returns specification tibble

Method define(): Define analysis specifications (create analysis objects)

Usage:

CohortPrevalenceExperiment\$define(autoIdStart = 1L)

Arguments:

autoIdStart Numeric. Starting ID for analyses (default: 1L)

Returns: List of CohortPrevalenceAnalysis objects with specification attached as attribute

Method clone(): The objects of this class are cloneable with this method.

Usage:

CohortPrevalenceExperiment\$clone(deep = FALSE)

Arguments:

deep Whether to make a deep clone.

createCohortPrevalenceAnalysis

Create a CohortPrevalenceAnalysis object

Description

Constructs an CohortPrevalenceAnalysis object with the specified settings.

Usage

```
createCohortPrevalenceAnalysis(
  analysisId,
  analysisTag = NULL,
  prevalentCohort,
  periodOfInterest,
  prevalenceType,
  useOnlyFirstObservationPeriod = FALSE,
  multiplier = 100000L,
  strata = NULL,
  demographicConstraints = createDemographicConstraints(),
  populationCohort = NULL,
  outputTypes = "prevalence",
  drugConceptSets = NULL
)
```

Arguments

<code>analysisId</code>	Unique integer <code>analysisId</code> to identify the analysis (required).
<code>prevalentCohort</code>	A <code>TargetCohort</code> object specifying the cohort of interest (required).
<code>periodOfInterest</code>	A <code>PeriodOfInterest</code> object (required).
<code>prevalenceType</code>	A <code>PrevalenceType</code> object (required). The lead-in requirement is carried by this object via <code>createPrevalenceType(leadInDays =)</code> .
<code>useOnlyFirstObservationPeriod</code>	Logical: TRUE to restrict analysis to the first observation period (optional).
<code>multiplier</code>	Integer specifying prevalence multiplier (optional).
<code>strata</code>	Character string. Must be one, or some of: "age", "gender", "race" (optional).
<code>demographicConstraints</code>	a <code>DemoConstraint</code> object specifying the constraints of the population.
<code>populationCohort</code>	A <code>PopulationCohort</code> object specifying the population of interest on which to compute prevalence.
<code>outputTypes</code>	Character vector specifying which output types to generate. Defaults to "prevalence". Can include "incidence" and/or "drugs" for simultaneous generation using shared base tables. Warning: The "drugs" output type is experimental and should be used with caution. Results may be subject to future changes.
<code>drugConceptSets</code>	Optional list of <code>Capr ConceptSetItems</code> . Required if "drugs" is in <code>outputTypes</code> , otherwise ignored.

Value

A `CohortPrevalenceAnalysis` R6 object.

```
createDemographicConstraints
```

Create a DemoConstraint object

Description

Constructs an DemoConstraint object for prevalence analyses.

Usage

```
createDemographicConstraints(
  ageMin = 0,
  ageMax = 150,
  genderIds = c(8507, 8532)
)
```

Arguments

ageMin	The minimum age allowed for the population. Default is 0
ageMax	the maximum age allowed for the population. Default is 150
genderIds	the genderIds allowed. Default is 8507 - M, and 8532 - F

Value

A DemoConstraint R6 object.

```
createDenominatorType
```

Create a DenominatorType object

Description

Constructs an DenominatorType object for denominator choice.

Usage

```
createDenominatorType(denomType, sufficientDays = NULL)
```

Arguments

denomType	Character string specifying denominator type. Must be one of: "pd1": Patients who have been observed on the first day of the period of interest "pd2": Patients who contribute all observable person-days in the period of interest. "pd3": Patients who contribute at least 1 day in the period of interest. "pd4": Patients who contribute sufficient time in the period of interest based on at least n observable person-days in the period of interest.
sufficientDays	Integer: For denominator choice "pd4", the number of minimum observable days patients must be observed.

Value

A DenominatorType R6 object.

`createPopulationCohort`

Create a population cohort PopulationCohort object

Description

Constructs a PopulationCohort object for the denominator population cohort. Population cohorts do not require a CIRCE JSON file or calculation mode.

Usage

```
createPopulationCohort(cohortId, cohortName)
```

Arguments

cohortId	Integer: the cohort ID within the database results schema of interest.
cohortName	Character string specifying a name for the cohort.

Value

A PopulationCohort R6 object.

`createPrevalenceType` *Create a PrevalenceType object*

Description

Constructs a PrevalenceType object specifying the prevalence type, lookback period, lead-in requirement, and calculation mode.

Usage

```
createPrevalenceType(  
  prevalenceType,  
  lookBackDays,  
  mode = "formal",  
  leadInDays = 0L,  
  sufficientDays = NULL  
)
```

Arguments

prevalenceType	Character string specifying prevalence type. Must be one of: • "point_prevalence": Status on a specific day (pn1 + pd1) • "period_prevalence_pd2": All time observed during period (pn2 + pd2) • "period_prevalence_pd3": Continuous observation (pn2 + pd3) • "period_prevalence_pd4": Sufficient days observed (pn2 + pd4)
lookBackDays	Integer number of days for lookback window. Must be an integer ≥ 0 or Inf for complete historical lookback. A value of 0 means no lookback — only events exactly on the interval day/period are counted.
mode	Character. Calculation mode controlling which cohort date anchors the event. Must be one of: • "formal" (default): Uses cohort_start_date as the anchor. Formal prevalence asks: did the disease START within the lookback window relative to the period of interest? • "rough": Uses cohort_end_date as the anchor. Rough prevalence asks: was the disease detected (still active) during the interval? This is a simpler detection-based calculation.
leadInDays	Integer number of days of continuous observation required before the start of each period of interest. Defaults to 0L (no lead-in requirement). Lead-in is a property of the prevalence definition rather than a shared experiment setting, so different prevalence types in the same experiment may use different values.
sufficientDays	Positive integer minimum days observed during the period of interest. Required when prevalenceType = "period_prevalence_pd4"; must be NULL for other types.

Value

A PrevalenceType R6 object.

```
createRassenIncidenceAnalysis
```

Create a IncidenceAnalysis object for Rassen Incidence

Description

Constructs an IncidenceAnalysis object with the specified settings.

Usage

```
createRassenIncidenceAnalysis(
  analysisId,
  targetCohort,
  periodOfInterest,
  minimumObservationLength = 0L,
  useOnlyFirstObservationPeriod = FALSE,
  multiplier = 100000L,
  strata = NULL,
  demographicConstraints = createDemographicConstraints(),
  populationCohort = NULL
)
```

Arguments

analysisId	Unique integer analysisId to identify the analysis (required).
targetCohort	A TargetCohort object specifying the cohort of interest (required).
periodOfInterest	A PeriodOfInterest object (required).
useOnlyFirstObservationPeriod	Logical: TRUE to restrict analysis to the first observation period (optional).
multiplier	Integer specifying prevalence multiplier (optional).
strata	Character string. Must be one, or some of: "age", "gender", "race" (optional).
demographicConstraints	a DemoConstraint object specifying the constraints of the population.
populationCohort	A CohortPopulation object specifying the population of interest on which to compute prevalence.
minimumObservationLength:	Integer specifying minimum observation length (optional).

Value

A IncidenceAnalysis R6 object.

createSpan	<i>Create a PeriodOfInterest object</i>
------------	---

Description

Constructs an PeriodOfInterest object for span prevalence analyses.

Usage

```
createSpan(startDates, endDates)
```

Arguments

startYears	A numeric vector of start years of interest.
endYears	A numeric vector of end years of interest.

Value

A PeriodOfInterest R6 object.

createStandardizationReference

Helper Functions for Standardization Reference Management

Description

Functions to create, import, and access standardization reference populations. Create a Standardization Reference from User Data

Usage

```
createStandardizationReference(
  name,
  country,
  year,
  source,
  data,
  reference = NULL
)
```

Arguments

name	Character. Name of the reference population
country	Character. Country or region
year	Integer. Reference year
source	Character. Data source description or citation
data	Data frame with columns: age, gender, population
reference	Character. URL or reference to access the source data (optional)

Details

The data frame should have one row per age-gender combination. Population values should be absolute counts (not proportions). Weights are automatically calculated as population / total_population.

Value

StandardizationReference object

Examples

```
## Not run:
my_data <- data.frame(
  age = c("0", "1-4", "5-9", "10-14", "0", "1-4", "5-9", "10-14"),
  gender = c(rep("Male", 4), rep("Female", 4)),
  population = c(100000, 400000, 420000, 410000, 95000, 380000, 400000, 405000)
)

my_ref <- createStandardizationReference(
  name = "Custom Census",
  country = "Fictional Country",
  year = 2020,
```

```

    source = "My Data Collection",
    data = my_data
  )

## End(Not run)

```

createTargetCohort	<i>Create a target cohort TargetCohort object</i>
--------------------	---

Description

Constructs a TargetCohort object for use as the prevalence numerator or incidence target cohort. All analyses use the era pattern (interval overlap).

Usage

```
createTargetCohort(cohortId, cohortName)
```

Arguments

cohortId	Integer: the cohort ID within the database results schema of interest.
cohortName	Character string specifying a name for the cohort.

Value

A TargetCohort R6 object.

createYearlyRange	<i>Create a PeriodOfInterest object</i>
-------------------	---

Description

Constructs an PeriodOfInterest object for yearly prevalence analyses.

Usage

```
createYearlyRange(range)
```

Arguments

range	A numeric vector of years of interest.
-------	--

Value

A PeriodOfInterest R6 object.

`exportPrevalenceQuery` *Export Prevalence Query*

Description

Exports the full SQL query of a CohortPrevalenceAnalysis analysis.

Usage

```
exportPrevalenceQuery(
  prevalenceAnalysisClass,
  executionSettings,
  outputFolder = NULL
)
```

Arguments

`prevalenceAnalysisClass` A CohortPrevalenceAnalysis R6 object with analysis settings (required).

`executionSettings` An ExecutionSettings R6 object with database details (required).

`outputFolder` Character string specifying the path to the folder where the output files will be saved. If left NULL, will default to current working directory (optional).

Value

Invisibly returns the rendered SQL string.

`fetchACSReference` *Fetch USA ACS Reference Population On-Demand*

Description

Fetch USA ACS Reference Population On-Demand

Usage

```
fetchACSReference(years, survey = "acs5", census_api_key = NULL)
```

Arguments

`years` Integer or vector of integers. ACS year(s) (e.g., 2020, or 2021:2023)

`survey` Character. Survey type: "acs5" (default) or "acs1"

`census_api_key` Character. Census API key. If NULL (default), uses CENSUS_API_KEY environment variable. Warns if neither provided.

Details

Fetches American Community Survey (ACS) estimates from Census API. Requires tidyCensus package and valid Census API key.

Note: ACS provides age-grouped estimates (not single-year). When multiple years are requested, data is combined into a single reference with year column.

Value

StandardizationReference object

Examples

```
## Not run:
# Option 1: Single year with environment variable key
# Sys.setenv(CENSUS_API_KEY = "your_key_here")
acs_2020 <- fetchACSReference(2020)

# Option 2: Single year with direct key parameter
acs_2020 <- fetchACSReference(2020, census_api_key = "your_key_here")

# Option 3: Multiple years (combined into one reference)
acs_2021_2023 <- fetchACSReference(2021:2023, survey = "acs5")

# Use in standardization
result <- standardizePrevalence(
  prevalenceData = my_data,
  referencePopulation = acs_2020
)

## End(Not run)
```

generatePrevalence	<i>Generate Prevalence Analysis Results</i>
--------------------	---

Description

Unified function for running one or more prevalence analyses with comprehensive result management. Replaces separate generateSinglePrevalence and generateMultiplePrevalence functions.

Usage

```
generatePrevalence(
  prevalenceAnalysisList,
  executionSettings,
  captureSql = TRUE
)
```

Arguments

prevalenceAnalysisList	A CohortPrevalenceAnalysis R6 object or list of such objects
executionSettings	An executionSettings R6 object with connection and schema details
captureSql	Logical. If TRUE (default), capture rendered SQL queries for audit trail

Details

This function consolidates analysis execution and result collection into a single workflow. Results include prevalence, incidence, and drug usage data as configured in the analysis objects. SQL queries are captured with SHA256 checksums for reproducibility verification.

When running multiple analyses from a CohortPrevalenceExperiment, use the experiment's `define()` method to obtain the analysis list, then pass it to this function:

```
exp <- CohortPrevalenceExperiment$new(...)$
  addCohorts(...)$
  addPrevalenceTypes(...)$
  # ... configure dimensions ...
  validate()

analyses <- exp$define()
results <- generatePrevalence(analyses, executionSettings)
```

Result Tracking:

- prevalence data frame: Main prevalence estimates
- incidence data frame: Incidence rates if requested
- drugUsage data frame: Drug usage patterns if requested
- metaInfo data frame: Analysis metadata and configuration

Query Audit Trail (Level 1):

Each executed query is captured and stored with SHA256 checksum in the `PrevalenceResults` object. Access via `results$show_query(analysisId)` for inspection.

Value

A `PrevalenceResults` R6 object containing all results with full provenance tracking

```
getStandardizationReference
```

Get Built-in Standardization Reference

Description

Get Built-in Standardization Reference

Usage

```
getStandardizationReference(name)
```

Arguments

name Character. Name of reference to retrieve. Valid options: "usa_census_2020", "japan_census_2020", "who_world_standard"

Value

StandardizationReference object

Examples

```
## Not run:
usa_ref <- getStandardizationReference("usa_census_2020")
japan_ref <- getStandardizationReference("japan_census_2020")
who_ref <- getStandardizationReference("who_world_standard")

## End(Not run)
```

importStandardizationReference

Import a Standardization Reference from CSV File

Description

Import a Standardization Reference from CSV File

Usage

```
importStandardizationReference(
  filepath,
  name,
  country,
  year,
  source,
  reference = NULL,
  encoding = "UTF-8"
)
```

Arguments

filepath Character. Path to CSV file

name Character. Name for the reference

country Character. Country or region

year Integer. Reference year

source Character. Data source description

reference Character. URL or reference to access the source data (optional)

encoding Character. File encoding (default: "UTF-8")

Details

CSV file should have columns: age, gender, population Additional columns are ignored.

Value

StandardizationReference object

Examples

```
## Not run:
my_ref <- importStandardizationReference(
  filepath = "data/my_population.csv",
  name = "Regional Census",
  country = "My Country",
  year = 2020,
  source = "Regional Statistics Bureau"
)

## End(Not run)
```

listStandardizationReferences

List Available Standardization References

Description

List Available Standardization References

Usage

```
listStandardizationReferences()
```

Details

Shows all built-in reference populations available in the package. Status indicates whether reference is active or deprecated.

Value

Data frame with columns: name, country, year, status

Examples

```
## Not run:
listStandardizationReferences()

## End(Not run)
```

`loadPrevalenceResults` *Load PrevalenceResults from Bundle*

Description

Load a `PrevalenceResults` object from an exported bundle directory containing CSV files and `manifest.json`

Usage

```
loadPrevalenceResults(bundlePath)
```

Arguments

`bundlePath` Character path to bundle directory

Value

`PrevalenceResults` object with data restored from bundle

`map_acs_b01001_to_age_sex`*Transform Raw ACS B01001 Data to Tidy Format*

Description

Takes raw output from `tidycensus::get_acs()` (B01001 table) and transforms it into tidy format with age, gender, population, and year columns.

Usage

```
map_acs_b01001_to_age_sex(acs_raw_df)
```

Arguments

`acs_raw_df` Data frame with columns: GEOID, NAME, variable, estimate, moe, year

- Typically output from `tidycensus::get_acs(table = "B01001", ...)`
- Must include year column

Details

Filters to only B01001 age-gender variables (excludes B01001_001 total and subtotals). Validates output: exactly 46 rows per year (23 age groups × 2 genders).

Value

Data frame with columns: age, gender, population, year (sorted by year, then gender, then age)

- age: Age group label
- gender: "Male" or "Female"
- population: Population estimate (numeric)
- weight: Population weight (population / total_population)
- year: Year of estimate

PopulationCohort

PopulationCohort R6 Class

Description

Describes a denominator population cohort for prevalence analyses. Does not require a CIRCE JSON file or a calculation mode.

Methods**Public methods:**

- [PopulationCohort\\$new\(\)](#)
- [PopulationCohort\\$id\(\)](#)
- [PopulationCohort\\$name\(\)](#)
- [PopulationCohort\\$viewCohortInfo\(\)](#)
- [PopulationCohort\\$clone\(\)](#)

Method `new()`: Create a new PopulationCohort object

Usage:

`PopulationCohort$new(id, name)`

Arguments:

`id` Integer cohort ID within the database results schema.

`name` Character cohort name.

Method `id()`: Return the cohort ID.

Usage:

`PopulationCohort$id()`

Method `name()`: Return the cohort name.

Usage:

`PopulationCohort$name()`

Method `viewCohortInfo()`: Return a formatted info string about this cohort.

Usage:

`PopulationCohort$viewCohortInfo()`

Method `clone()`: The objects of this class are cloneable with this method.

Usage:

`PopulationCohort$clone(deep = FALSE)`

Arguments:

`deep` Whether to make a deep clone.

PrevalenceResults	<i>PrevalenceResults R6 Class</i>
-------------------	-----------------------------------

Description

Container for prevalence analysis results with standardization capabilities. Manages export/import with full provenance tracking via manifest file.

Details

Results are stored as CSV files in a directory bundle with a manifest.json file tracking provenance, standardization parameters, and execution metadata.

Active Fields:

- prevalence: Data frame with crude (unadjusted) prevalence results (read/write)
- stdPrev: Data frame with standardized prevalence results (read/write)
- incidence: Data frame with incidence results (read/write)
- drugUsage: Data frame with drug usage results (read/write)
- metaInfo: Data frame with analysis metadata (read/write)
- standardizationApplied: List containing standardization parameters (read-only)

Private Tracking Fields (visible in print/summary only):

- executionId: Unique execution identifier
- exportDate: Timestamp of last export

Methods:

- initialize(): Create new PrevalenceResults object
- export(): Save results to directory bundle with manifest
- standardizePrevalence(): Apply direct method standardization (returns new object)
- validate(): Check data integrity and consistency
- summary(): Print summary statistics
- print(): Display object overview

Active bindings

prevalence Data frame with prevalence results

incidence Data frame with incidence results

drugUsage Data frame with drug usage results

metaInfo Data frame with analysis metadata

stdPrev Data frame with standardized prevalence results

standardizationApplied List of standardization parameters (read-only)

Methods

Public methods:

- [PrevalenceResults\\$new\(\)](#)
- [PrevalenceResults\\$export\(\)](#)
- [PrevalenceResults\\$standardizePrevalence\(\)](#)
- [PrevalenceResults\\$validate\(\)](#)
- [PrevalenceResults\\$summary\(\)](#)
- [PrevalenceResults\\$print\(\)](#)
- [PrevalenceResults\\$show_query\(\)](#)
- [PrevalenceResults\\$.addExecutedQuery\(\)](#)
- [PrevalenceResults\\$clone\(\)](#)

Method `new()`: Create new PrevalenceResults object

Usage:

```
PrevalenceResults$new(
  prevalence = NULL,
  incidence = NULL,
  drugUsage = NULL,
  metaInfo = NULL,
  executionId = NULL
)
```

Arguments:

`prevalence` Data frame with prevalence results

`incidence` Data frame with incidence results

`drugUsage` Data frame with drug usage results

`metaInfo` Data frame with analysis metadata

`executionId` Optional character string for execution tracking (internal use)

Method `export()`: Export results to directory bundle with manifest

Usage:

```
PrevalenceResults$export(outputFolder, bundleName = NULL)
```

Arguments:

`outputFolder` Character path where bundle directory will be created

`bundleName` Optional name for the bundle directory. Default: auto-generated timestamp

Returns: Invisibly returns self for chaining

Method `standardizePrevalence()`: Apply direct method standardization

Usage:

```
PrevalenceResults$standardizePrevalence(
  referencePopulation,
  ageMin = NULL,
  ageMax = NULL,
  ageRightTruncation = NULL
)
```

Arguments:

`referencePopulation` StandardizationReference object

ageMin Numeric minimum age for filtering
ageMax Numeric maximum age for filtering
ageRightTruncation Numeric age threshold for collapsing
Returns: New PrevalenceResults object with standardized prevalence

Method validate(): Validate data integrity and relationships

Usage:
PrevalenceResults\$validate()
Returns: Logical TRUE if valid, otherwise stops with error

Method summary(): Print summary of results

Usage:
PrevalenceResults\$summary()
Returns: Invisibly returns self

Method print(): Print object

Usage:
PrevalenceResults\$print()
Returns: Invisibly returns self

Method show_query(): Display executed SQL query for an analysis

Usage:
PrevalenceResults\$show_query(analysisId)
Arguments:
analysisId Character or numeric analysis ID
Returns: Invisibly returns the SQL string

Method .addExecutedQuery(): Add executed query for an analysis (internal use)

Usage:
PrevalenceResults\$.addExecutedQuery(analysisId, sql_string)
Arguments:
analysisId Character or numeric analysis ID
sql_string Character SQL query string
Returns: Invisibly returns self

Method clone(): The objects of this class are cloneable with this method.

Usage:
PrevalenceResults\$clone(deep = FALSE)
Arguments:
deep Whether to make a deep clone.

StandardizationReference

StandardizationReference R6 Class

Description

Encapsulates a reference population for direct method standardization. Stores population weights by age and gender with metadata.

Details

The StandardizationReference class provides a structured way to manage reference populations used for age-sex standardization. It validates data structure, auto-calculates weights as proportions, and provides methods for accessing and manipulating reference data.

Active bindings

name Name of the reference population (active)
country Country or region (active)
year Year of reference (numeric) (active)
source Source/citation (active)
reference URL or reference to access source data (active)

Methods

Public methods:

- [StandardizationReference\\$new\(\)](#)
- [StandardizationReference\\$viewReference\(\)](#)
- [StandardizationReference\\$getData\(\)](#)
- [StandardizationReference\\$getTotalPopulation\(\)](#)
- [StandardizationReference\\$getAgeValues\(\)](#)
- [StandardizationReference\\$getGenderValues\(\)](#)
- [StandardizationReference\\$getMetadata\(\)](#)
- [StandardizationReference\\$getFilteredReference\(\)](#)
- [StandardizationReference\\$getAdjustedReference\(\)](#)
- [StandardizationReference\\$validateRightTruncation\(\)](#)
- [StandardizationReference\\$getValidTruncationPoints\(\)](#)
- [StandardizationReference\\$mapAgesToReference\(\)](#)
- [StandardizationReference\\$clone\(\)](#)

Method new(): Create a new StandardizationReference

Usage:

```
StandardizationReference$new(
  name,
  country,
  year,
  source,
```

```

    data,
    reference = NULL
)

```

Arguments:

name Character. Name of reference (e.g., "USA Census 2020")
country Character. Country/region name
year Integer. Reference year
source Character. Data source description
data Data frame with columns: age, gender, population
reference Character. URL or reference to access the source data

Returns: A new StandardizationReference object

Method viewReference(): View the reference population data

Usage:

```
StandardizationReference$viewReference()
```

Method getData(): Get the full reference data frame

Usage:

```
StandardizationReference$getData()
```

Returns: Data frame with columns: age, gender, population, weight

Method getTotalPopulation(): Get total population

Usage:

```
StandardizationReference$getTotalPopulation()
```

Returns: Numeric value

Method getAgeValues(): Get unique ages

Usage:

```
StandardizationReference$getAgeValues()
```

Returns: Character vector of age values

Method getGenderValues(): Get unique genders

Usage:

```
StandardizationReference$getGenderValues()
```

Returns: Character vector of gender values

Method getMetadata(): Get metadata as list

Usage:

```
StandardizationReference$getMetadata()
```

Returns: List with name, country, year, source, reference

Method getFilteredReference(): Filter reference to demographic bounds and re-normalize weights

Usage:

```
StandardizationReference$getFilteredReference(ageMin = NULL, ageMax = NULL)
```

Arguments:

ageMin Minimum age (inclusive)

ageMax Maximum age (inclusive)

Returns: Data frame filtered and re-normalized to bounds

Method getAdjustedReference(): Apply both bounds filtering and age truncation

Usage:

```
StandardizationReference$getAdjustedReference(
  ageMin = NULL,
  ageMax = NULL,
  rightTruncation = NULL
)
```

Arguments:

ageMin Minimum age (inclusive)

ageMax Maximum age (inclusive)

rightTruncation Age threshold for truncation (optional)

Returns: Data frame with both transformations applied

Method validateRightTruncation(): Validate right truncation point against reference age groups

For single-year references, any integer age is valid. For grouped references, the truncation point must fall exactly at the start of an age group (e.g., "85" for "85+" group, "80" for "80-84" group). Fails fast if truncation falls mid-group.

Usage:

```
StandardizationReference$validateRightTruncation(rightTruncation)
```

Arguments:

rightTruncation Numeric. Age threshold to validate.

Returns: Numeric. The validated truncation point (or stop with error)

Method getValidTruncationPoints(): Get valid truncation points for this reference

Returns a numeric vector of all valid age thresholds where right truncation is allowed (start of each age group for grouped references, all ages for single-year references).

Usage:

```
StandardizationReference$getValidTruncationPoints()
```

Returns: Numeric vector of valid truncation points

Method mapAgesToReference(): Map single-year ages to reference age group labels

Converts a character/numeric vector of ages (some possibly "N+" labels from right truncation) to the reference's age label format so prevalence data can be joined to reference weights by (age, gender).

Works generically for any reference type:

- Already-truncated "N+" values pass through unchanged
- Single-year references (Decennial Census): zero-pad numeric values
- Grouped references (ACS, WHO): lookup which group each numeric age falls into

Usage:

```
StandardizationReference$mapAgesToReference(age_values)
```

Arguments:

`age_values` Character/numeric vector. May contain `\N+` suffix values or numeric strings.

Returns: Character vector of reference-formatted age labels

Method `clone()`: The objects of this class are cloneable with this method.

Usage:

```
StandardizationReference$clone(deep = FALSE)
```

Arguments:

`deep` Whether to make a deep clone.

Examples

```
## Not run:
# Create a custom reference
ref_data <- data.frame(
  age = c("0", "1", "2", "100+", "0", "1", "2", "100+"),
  gender = c(rep("Male", 4), rep("Female", 4)),
  population = c(100000, 95000, 92000, 50000, 98000, 93000, 90000, 48000)
)

my_ref <- StandardizationReference$new(
  name = "Custom Reference",
  country = "Custom",
  year = 2020L,
  source = "My Data",
  data = ref_data
)

# View reference
my_ref$viewReference()

# Get total population
my_ref$getTotalPopulation()

# Filter to age range
filtered <- my_ref$getFilteredReference(ageMin = 18, ageMax = 65)

## End(Not run)
```

TargetCohort

TargetCohort R6 Class

Description

Describes a study cohort used as the numerator (prevalent) or target (incidence) in an analysis. Uses the era pattern (interval overlap) for all analyses.

Methods

Public methods:

- [TargetCohort\\$new\(\)](#)
- [TargetCohort\\$id\(\)](#)

- [TargetCohort\\$name\(\)](#)
- [TargetCohort\\$viewCohortInfo\(\)](#)
- [TargetCohort\\$clone\(\)](#)

Method `new()`: Create a new TargetCohort object

Usage:

`TargetCohort$new(id, name)`

Arguments:

`id` Integer cohort ID within the database results schema.

`name` Character cohort name.

Method `id()`: Return the cohort ID.

Usage:

`TargetCohort$id()`

Method `name()`: Return the cohort name.

Usage:

`TargetCohort$name()`

Method `viewCohortInfo()`: Return a formatted info string about this cohort.

Usage:

`TargetCohort$viewCohortInfo()`

Method `clone()`: The objects of this class are cloneable with this method.

Usage:

`TargetCohort$clone(deep = FALSE)`

Arguments:

`deep` Whether to make a deep clone.

Index

`acs_age_groups`, [2](#)

`CohortPrevalenceExperiment`, [3](#)
`createCohortPrevalenceAnalysis`, [6](#)
`createDemographicConstraints`, [8](#)
`createDenominatorType`, [8](#)
`createPopulationCohort`, [9](#)
`createPrevalenceType`, [9](#)
`createRassenIncidenceAnalysis`, [10](#)
`createSpan`, [11](#)
`createStandardizationReference`, [12](#)
`createTargetCohort`, [13](#)
`createYearlyRange`, [13](#)

`exportPrevalenceQuery`, [14](#)

`fetchACSReference`, [14](#)

`generatePrevalence`, [15](#)
`getStandardizationReference`, [16](#)

`importStandardizationReference`, [17](#)

`listStandardizationReferences`, [18](#)
`loadPrevalenceResults`, [19](#)

`map_acs_b01001_to_age_sex`, [19](#)

`PopulationCohort`, [20](#)
`PrevalenceResults`, [21](#)

`StandardizationReference`, [24](#)

`TargetCohort`, [27](#)